Feature Engineering (Author = Miraj Deep Bhandari)

Definition

> **Feature engineering** is the process of using **domain knowledge** to extract, transform, or create features from raw data in order to improve the performance of machine learning algorithms.

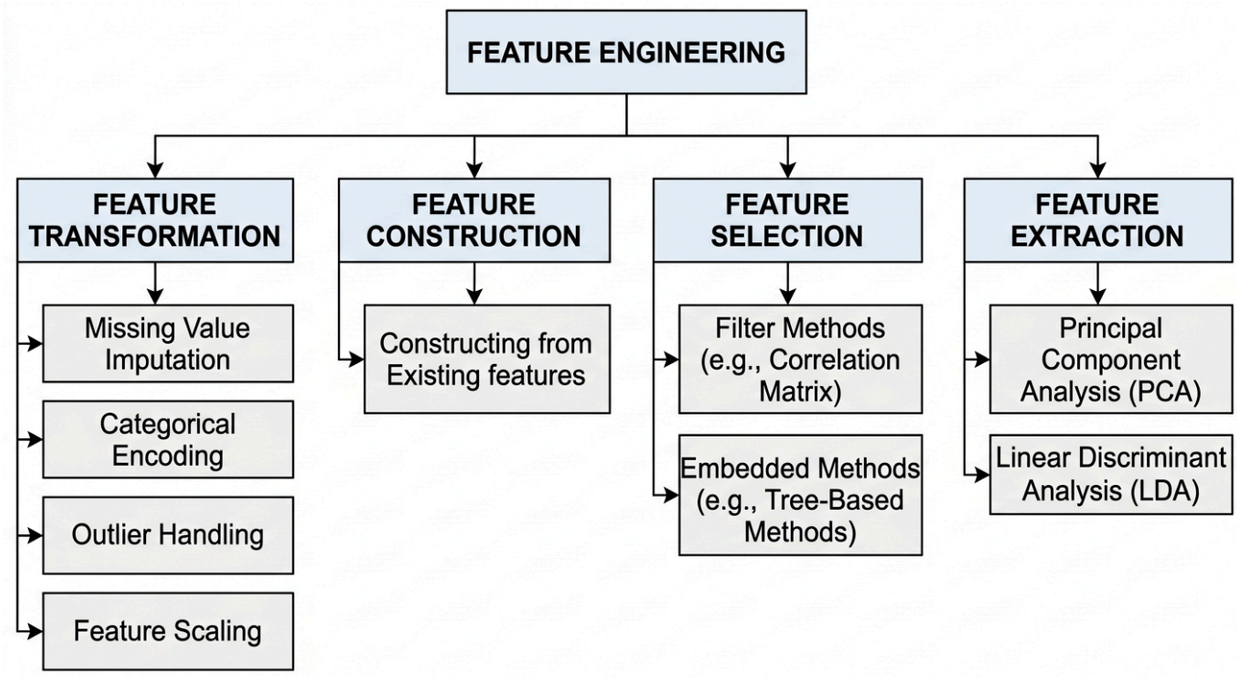
Raw data collected from the real world is rarely in a form that can be directly fed into a machine learning model. It needs to be shaped, cleaned, enriched, and structured. This entire process is called feature engineering.

Feature Engineering in the ML Pipeline

```
Raw Data
   ↓
Data Understanding / Descriptive
   ↓
Exploratory Data Analysis (EDA)
   ↓
Feature Engineering          ← This entire stage
  ├── Feature Transformation
  ├── Feature Construction
  ├── Feature Selection
  └── Feature Extraction
   ↓
Model Training
   ↓
Evaluation & Tuning
   ↓
Deployment
```

Feature scaling and feature selection are sub-tasks that fall under the broader umbrella of feature engineering.

The 4 Types of Feature Engineering



---

1. Feature Transformation

You take an **existing feature (column)** in the dataset and convert it into a more useful or model-compatible format. Nothing new is created — the existing data is reshaped.

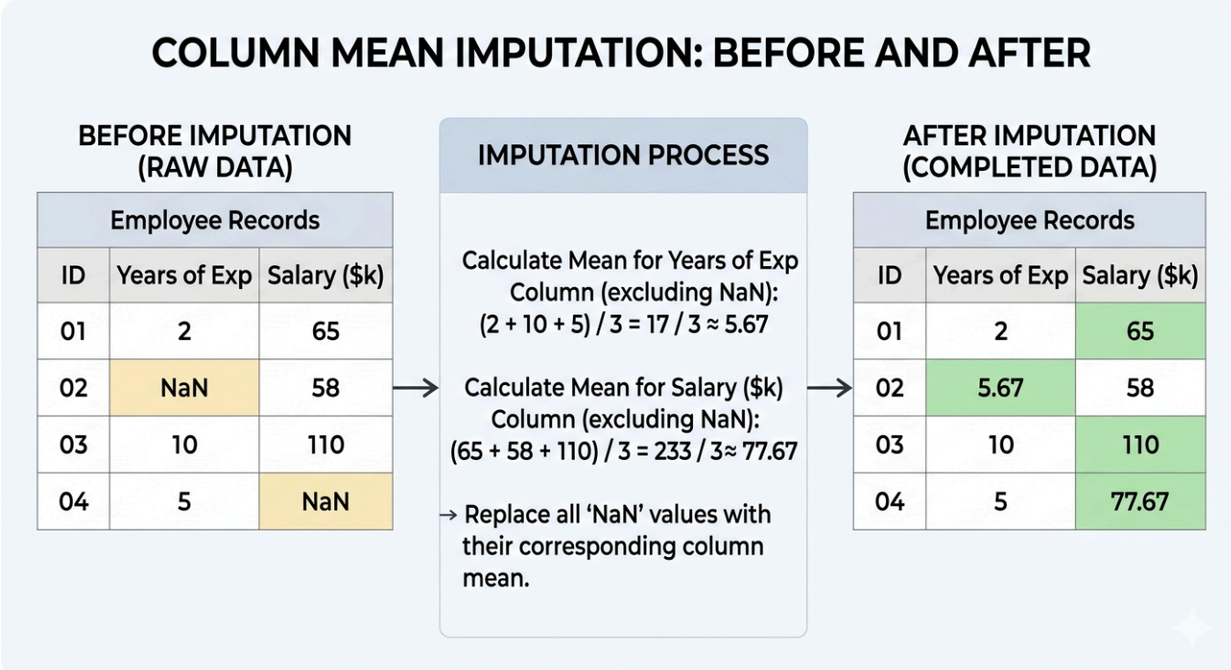
1.1 Missing Value Imputation

Real-world datasets almost always contain missing values due to data collection errors, sensor failures, human omission, or privacy filtering.

Why it matters: Most ML algorithms cannot handle `NaN` values directly and will either throw errors or produce unreliable results.

Options for handling missing values:

Strategy	When to Use
Drop rows	When missing values are very few (< 1–2%) and random
Drop column	When a feature has > 70–80% missing values
Mean imputation	For numerical data with no significant outliers (normal distribution)
Median imputation	For numerical data with outliers or skewed distribution
Mode imputation	For categorical data
Forward/Backward fill	For time-series data
KNN Imputation	When relationships between features can predict missing values
Model-based imputation	Train a regression/classification model to predict the missing values



1.2 Categorical Encoding

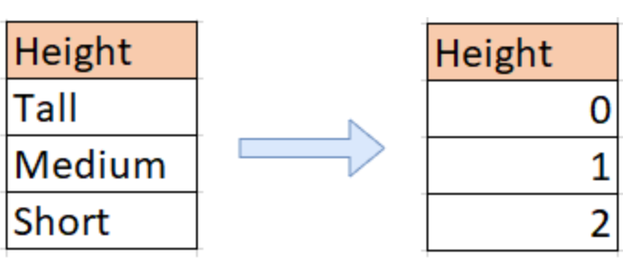
Machine learning models are mathematical in nature — they work with numbers, not text labels. Categorical columns (like `City`, `Gender`, `Product Type`) must be converted to numerical representations.

Types of categorical variables:

- **Nominal** — no inherent order (e.g. `Red`, `Blue`, `Green`)
- **Ordinal** — has a meaningful order (e.g. `Low`, `Medium`, `High`)

Encoding techniques:

Technique	Best For	Notes
Label Encoding	Ordinal categories	Assigns integer 0, 1, 2...
One-Hot Encoding	Nominal categories (low cardinality)	Creates a binary column per category can cause the dummy variable trap
Ordinal Encoding	Ordinal categories	Manual mapping that preserves order



Color
Red
Green
Blue
Green

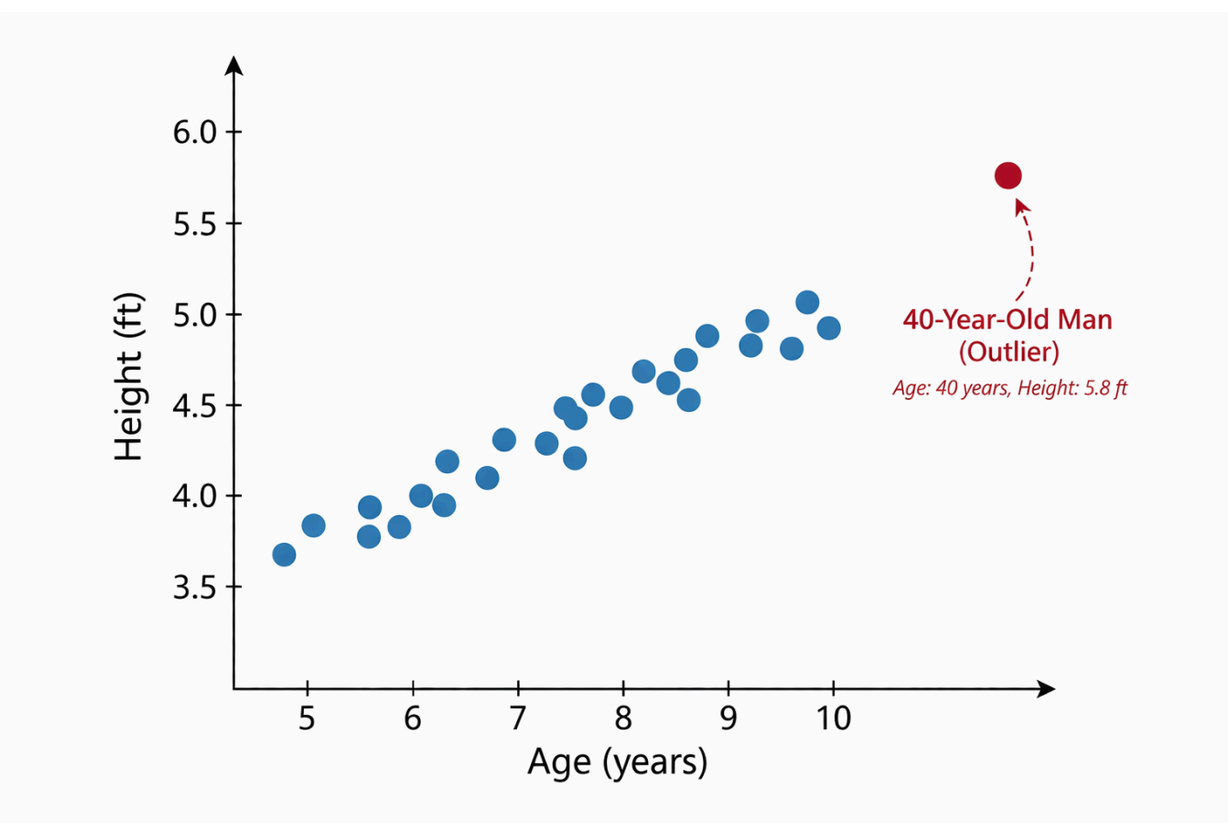
→

Red	Green	Blue
1	0	0
0	1	0
0	0	1
0	1	0

> **Dummy Variable Trap:** When using one-hot encoding, always drop one column (n−1 columns for n categories) to avoid perfect multicollinearity.

1.3 Outlier Detection

An **outlier** is a data point that deviates significantly from the rest of the distribution. Outliers can distort statistical measures (mean, variance) and degrade model performance, especially for linear models.



1.3.3 Mean without the Outlier

Example ages:

2, 3, 4, 5, 6, 7, 8, 9, 10

Mean formula

Mean = (Sum of values) / (Number of values)

Sum = 54
Count = 9

Mean = 54 / 9 = **6**

So the **average age = 6 years**.

1.3.4 Mean with the Outlier (40-year-old)

Now include the **40-year-old man**.

Data:

2, 3, 4, 5, 6, 7, 8, 9, 10, 40

Sum = 94
Count = 10

Mean = 94 / 10 = **9.4**

1.3.5 Impact of the Outlier

Case	Mean
Without outlier	6
With outlier (40)	9.4

Detection methods:

Method	How It Works
Z-Score	Points beyond ±3 standard deviations from the mean
IQR Method	Points below Q1 − 1.5×IQR or above Q3 + 1.5×IQR
Box Plot	Visual inspection of whisker boundaries

Treatment strategies:

Strategy	Description
Remove	Drop the outlier rows (only if data is large and outliers are few)
Cap / Winsorize	Replace outliers with the boundary value (e.g. 95th percentile)
Transform	Apply log/sqrt transform to compress the range
Keep	In some domains (fraud, medical), outliers ARE the signal — do not remove

---

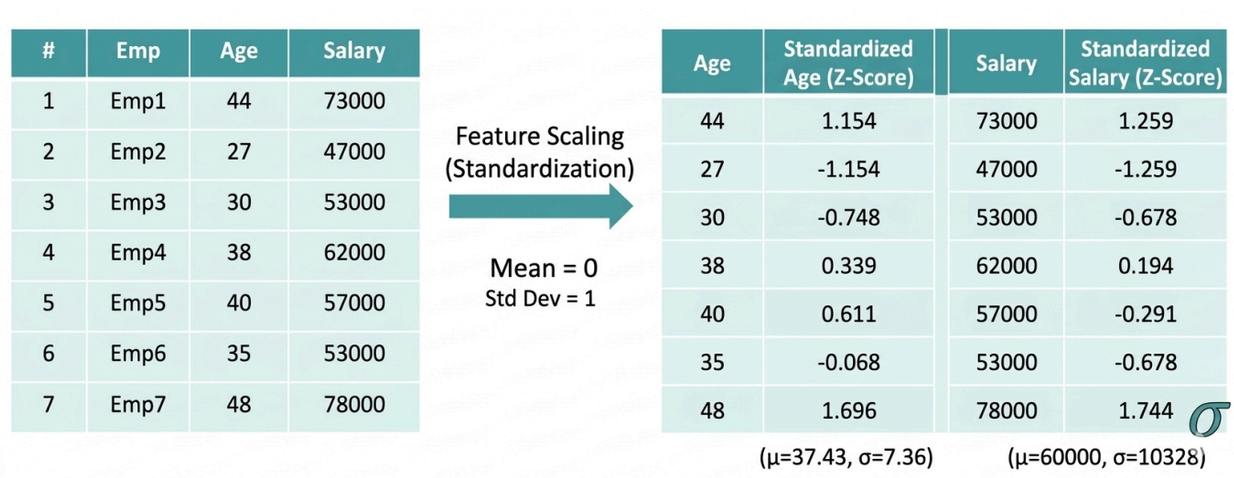
1.4 Feature Scaling

When features have vastly different numerical ranges (e.g. `age` : 0–100 vs `salary` : 10,000–500,000), distance-based and gradient-based models will be dominated by the larger-scale feature.

Feature scaling ensures all features contribute **proportionally**.

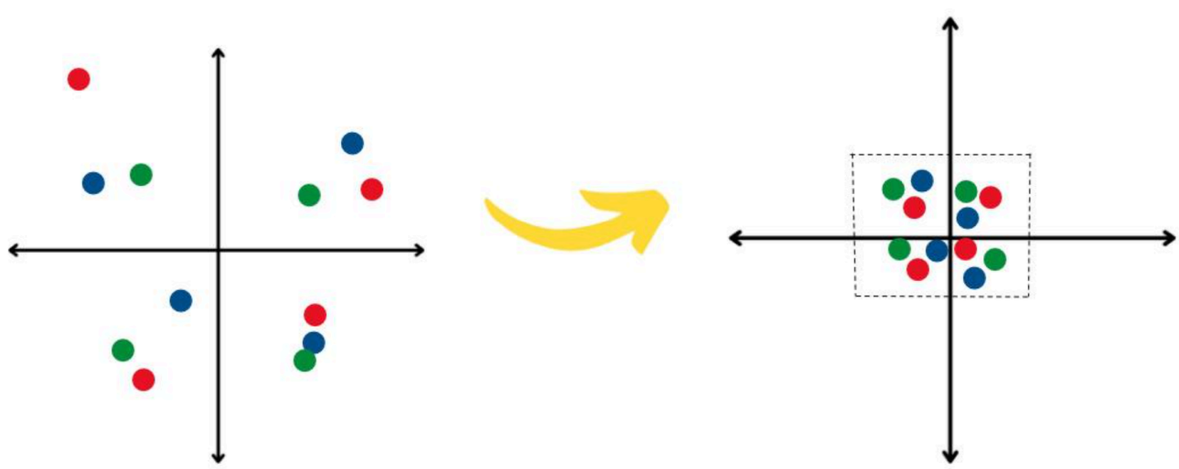
Scaling techniques:

Technique	Formula	Output Range	Best For
Min-Max Normalization	$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$	[0, 1]	When distribution is not Gaussian, neural networks
Standardization (Z-score)	$x' = \frac{x - \mu}{\sigma}$	Mean=0, Std=1	When distribution is approximately Gaussian, SVM, logistic regression
Robust Scaling	$x' = \frac{x - Q_2}{Q_3 - Q_1}$	Centered on median	When data has significant outliers



#	Emp	Age	Salary
1	Emp1	44	73000
2	Emp2	27	47000
3	Emp3	30	53000
4	Emp4	38	62000
5	Emp5	40	57000
6	Emp6	35	53000
7	Emp7	48	78000

Age	Standardized Age (Z-Score)	Salary	Standardized Salary (Z-Score)
44	1.154	73000	1.259
27	-1.154	47000	-1.259
30	-0.748	53000	-0.678
38	0.339	62000	0.194
40	0.611	57000	-0.291
35	-0.068	53000	-0.678
48	1.696	78000	1.744



> **Important:** Models that **do not** need scaling: Decision Trees, Random Forest, XGBoost (tree-based models are invariant to monotonic transformations).
> Models that **do** need scaling: Linear Regression, Logistic Regression, SVM, KNN, Neural Networks, PCA.

---

2. Feature Construction

Feature construction means **creating entirely new features** that do not exist in the raw dataset, derived from one or more existing columns using domain knowledge, mathematical operations, or logical combinations.

This is the most creative aspect of feature engineering and depends heavily on **domain expertise** and **intuition**.

Before: Raw Physical Attributes

ID	Age	Height (m)	Gender
1	25	1.80	Male
2	34	1.65	Female
3	45	1.75	Male
4	29	1.70	Female

After: Constructed BMI Dataset

ID	Age	BMI	Gender
1	25	**24.69**	Male
2	34	**20.20**	Female
3	45	**32.65**	Male
4	29	**24.91**	Female

Note: BMI = weight (kg) ÷ height^2 (m^2)

---

3. Feature Selection

Feature selection is the process of **identifying and keeping only the most relevant features** while discarding those that are redundant, irrelevant, or harmful to model performance.

> More features ≠ better model. Irrelevant features introduce noise, increase training time, and can cause overfitting.

Why Feature Selection Matters

- Reduces **overfitting** by removing noise variables
- Improves **training speed** and reduces memory usage
- Increases **model interpretability**
- Can improve **generalization** (test set performance)

Methods For Feature Selection

Method	Measures	Best For
Pearson Correlation	Linear relationship between feature and target	Continuous target
Spearman Correlation	Monotonic relationship	Non-linear, ranked
Chi-Square Test	Statistical dependency between categorical variables	Classification
ANOVA F-test	Variance between groups	Numerical features, categorical target
Tree-based Feature Importance	Feature contribution based on impurity reduction (Gini/MSE)	Non-linear relationships (Random Forest, GBM)
Lasso (L1 Regularization)	Shrinks some coefficients to zero (automatic feature removal)	High-dimensional data, sparse feature selectio

Method	Measures	Best For
Ridge (L2 Regularization)	Shrinks coefficients but does not eliminate features	Multicollinearity handling
Elastic Net	Combination of L1 and L2 regularization	When features are highly correlated

---

3.4 Dimensionality Considerations

- **Curse of Dimensionality:** As the number of features grows, data becomes increasingly sparse in the feature space — models require exponentially more data to generalize well
- **Multicollinearity:** Highly correlated features carry redundant information, removing one improves model stability

---

4. Feature Extraction

Feature extraction creates **new, transformed features** from the existing ones — often reducing the number of dimensions while retaining as much information as possible. Unlike feature construction (which uses domain knowledge), extraction uses mathematical algorithms to find new representations of the data.

4.1 Principal Component Analysis (PCA)

PCA is an **unsupervised** linear dimensionality reduction technique. It finds the directions (principal components) of maximum variance in the data and projects it onto a lower-dimensional subspace.

How it works:

1. Standardize the data
2. Compute the covariance matrix
3. Compute eigenvectors and eigenvalues
4. Sort eigenvectors by decreasing eigenvalue
5. Select top k eigenvectors → these are the principal components
6. Project data onto the new k-dimensional space

$$Z = X \cdot W_k$$

Where W_k is the matrix of the top k eigenvectors.

Key properties:

- Principal components are **orthogonal** (uncorrelated)
- The first PC captures the most variance, the second the next most, etc.
- **Explained variance ratio** tells you how much information each component carries
- Information loss is inevitable — choose k to retain ≥ 95% explained variance

Best for: High-dimensional data (images, text embeddings, sensor arrays), removing multicollinearity, visualization (reduce to 2D/3D)

Limitation: Components are linear combinations of original features — **not interpretable**

4.2 Linear Discriminant Analysis (LDA)

LDA is a **supervised** linear dimensionality reduction technique that finds the axes that **maximize class separability** rather than variance.

Objective: Find projection that maximizes:

$$J(W) = \frac{W^T S_B W}{W^T S_W W}$$

Where:

- S_B = Between-class scatter matrix
- S_W = Within-class scatter matrix

Key difference from PCA:

	PCA	LDA
Type	Unsupervised	Supervised
Objective	Maximize variance	Maximize class separation
Max components	min(n_samples, n_features) − 1	n_classes − 1
Uses labels	No	Yes

Best for: Classification problems where class discrimination is important

---

Summary Comparison

Type	Input	Output	Requires Domain Knowledge	Use Case
Transformation	Existing features	Modified same features	Low	Cleaning, scaling, encoding raw data
Construction	Existing features	New additional features	High	Adding meaningful derived signals
Selection	All features	Subset of features	Medium	Reducing noise, improving efficiency
Extraction	All features	New transformed feature space	Low	Dimensionality reduction, compression

---

Common Feature Engineering Mistakes

Mistake	Consequence	Fix
Scaling before train/test split	Data leakage from test set into training	Fit scaler on train set only, transform both
Imputing before split	Test data statistics leak into training	Same — fit imputer on train only
One-hot encoding without dropping one column	Multicollinearity (dummy variable trap)	Use `drop_first=True`
Removing outliers blindly	May delete genuine rare events	Understand context before removing
Applying PCA before feature selection	PCA on noisy features = noisy components	Select relevant features first

---

Practical Workflow

```
1. Understand the data (EDA)
        ↓
2. Handle missing values
        ↓
3. Encode categorical variables
        ↓
4. Detect and treat outliers
        ↓
5. Construct new features (domain knowledge)
        ↓
6. Scale numerical features
        ↓
7. Select relevant features
        ↓
8. Extract compressed representations (if needed)
        ↓
9. Feed into ML model
```